UNIX Revision Notes

Row Selection, Column Selection & Merge Files

1. ROW SELECTION (SELECT LINES)

`head`

Purpose: Prints the first few lines of a file.

Syntax

```
head -n number filename
```

Example

```
head -n 3 student.txt
```

Prints the first 3 lines.

`tail`

Purpose: Prints the last few lines of a file.

Syntax

```
tail -n number filename
```

Example

```
tail -n 2 student.txt
```

Prints the last 2 lines.

`sed`

Purpose: Prints specific lines.

Syntax

```
sed -n 'start,endp' filename
```

Examples

```
sed -n '2,5p' student.txt
```

Prints lines 2 to 5.

```
sed -n '3p' student.txt
```

Prints only line 3.

Note: `-n` disables automatic printing. `p` prints the selected lines.

2. COLUMN SELECTION – `CUT`

Purpose: Extracts characters or fields (columns) from a file.

Flags

`-d`

Delimiter (separator between fields)

Examples:

Space separated:

```
cut -d " " -f2 student.txt
```

Comma separated:

```
cut -d "," -f2 student.txt
```

Colon separated:

```
cut -d ":" -f2 student.txt
```

`-f`

Field (Column) Selection

Examples:

```
cut -d "," -f2 student.txt
```

Prints field 2.

```
cut -d "," -f2,3 student.txt
```

Prints fields 2 and 3.

```
cut -d "," -f1,3,4 student.txt
```

Prints fields 1, 3 and 4.

```
cut -d "," -f2-
```

Prints from field 2 to the last field.

`-c`

Character Selection

Examples:

```
cut -c1-5 names.txt
```

Prints characters 1 to 5.

```
cut -c1 names.txt
```

Prints only the first character.

```
cut -c3-7 names.txt
```

Prints characters 3 to 7.

> ## Difference Between `-f` and `-c`
>
> `-f` → Selects **fields (columns)**

```
101,Indra,A,25
```

```
cut -d "," -f2
```

Output: `Indra`

`-c` → Selects **character positions**

```
cut -c1-10 student.txt
```

Output: `101,Indra,`

3. ROW + COLUMN SELECTION TOGETHER

Select rows first using `sed`, then columns using `cut`.

Syntax

```
sed -n 'start,endp' filename | cut -d "," -ffields
```

Example

```
sed -n '1,3p' student.txt | cut -d "," -f2,3
```

Output:

```
Indra,A
Rahul,B
Amit,A
```

4. MERGE FILES HORIZONTALLY – `PASTE`

Purpose: Merges corresponding lines from two or more files.

Syntax

Merge two files

```
paste file1.txt file2.txt
```

Merge using comma

```
paste -d "," file1.txt file2.txt
```

Example

file1.txt

```
101
102
103
```

file2.txt

```
Indra
Rahul
Amit
```

Command

```
paste -d "," file1.txt file2.txt
```

Output

```
101,Indra
102,Rahul
103,Amit
```

5. COMBINE `CUT` + `PASTE`

Suppose:

file1.txt

```
101,Indra,A,25
102,Rahul,B,18
103,Amit,A,12
```

file2.txt

```
12-01-2026,9876543210
15-01-2026,9988776655
18-01-2026,9123456789
```

Command

```
cut -d "," -f1,2 file1.txt | paste -d "," - file2.txt
```

Output

```
101,Indra,12-01-2026,9876543210
102,Rahul,15-01-2026,9988776655
103,Amit,18-01-2026,9123456789
```

Note: The `-` in `paste` means **take input from the previous command (pipe/standard input).**

QUICK REVISION SUMMARY

Row Selection

Command	Purpose
`head -n`	First N lines
`tail -n`	Last N lines
`sed -n '2,5p'`	Specific rows

`cut` Flags

Flag	Meaning
`-d`	Delimiter (separator)
`-f`	Fields (columns)
`-c`	Character positions

`paste`

Command	Purpose
`paste file1 file2`	Merge files horizontally
`paste -d "," file1 file2`	Merge using comma

`paste -d "," - file2`	Merge piped output with a file

> **Memory Tricks**
>
> - **Rows →** `head`, `tail`, `sed`
> - **Columns →** `cut`
> - **Merge →** `paste`
> - `-d` → Delimiter
> - `-f` → Fields
> - `-c` → Characters
> - `-n` **(sed)** → No automatic printing
> - `p` → Print selected lines
> - `-` **in** `paste` → Read input from the pipe

4. SPLIT LARGE FILES INTO SMALLER FILES

Command: `split`

Work: Breaks a large file into multiple smaller files.

```
split -l 100 bigfile.txt part_
```

Splits bigfile.txt into chunks of 100 lines each, naming output files part_aa, part_ab, etc.

```
split -b 1m bigfile.txt chunk_
```

The `-b` flag splits by size instead of lines — here, 1 MB per chunk.

5. COUNTING LINES, WORDS, CHARACTERS

Command: `wc` (word count)

```
wc file.txt
```

Outputs three numbers: lines, words, characters — in that order.

```
wc -l file.txt
```

Shows only the line count.

```
wc -w file.txt
```

Shows only the word count.

```
wc -c file.txt
```

Shows only the character (byte) count.

6. SORTING CONTENT BY SPECIFIC COLUMN

Command: `sort`

```
sort -k2 file.txt
```

Sorts lines based on the 2nd field/column (`-k2`), using whitespace as the default separator.

```
sort -t: -k3 /etc/passwd
```

The `-t:` sets the field separator to `:`, then sorts by the 3rd field — useful for files like /etc/passwd which are colon-delimited.

7. NUMERICAL SORTING

Command: `sort -n`

Work: Sorts numbers by actual numeric value, not as plain text (since default text-sort would put "10" before "9").

```
sort -n numbers.txt
```

Sorts the file's content in proper numeric order (1, 2, 9, 10...) instead of alphabetical/lexical order (1, 10, 2, 9...).

```
sort -nr numbers.txt
```

The `-r` flag combined with `-n` sorts numerically in reverse (descending) order.

8. COMPARING TWO FILES LINE BY LINE

Command: `diff`

Work: Compares two files and shows the differences line by line.

```
diff file1.txt file2.txt
```

Shows which lines differ between the two files, using `<` for lines only in file1.txt and `>` for lines only in file2.txt.

```
diff -u file1.txt file2.txt
```

The `-u` flag shows differences in unified format — a more compact, readable diff style commonly seen in patches.

9. COMPARING TWO FILES CHARACTER BY CHARACTER

Command: `cmp`

Work: Compares two files byte by byte and reports the first point of difference — much more granular than diff.

```
cmp file1.txt file2.txt
```

Reports the first byte and line number where the two files differ. If identical, it outputs nothing.

```
cmp -l file1.txt file2.txt
```

The `-l` flag lists every byte position that differs, along with the differing byte values (in octal).

10. COMPARING TWO SORTED FILES

Command: `comm`

Work: Compares two already-sorted files and shows lines unique to each, and lines common to both — organized into 3 columns.

```
comm file1.txt file2.txt
```

Outputs 3 columns: Column 1 = lines only in file1.txt, Column 2 = lines only in file2.txt, Column 3 = lines common to both. (Note: both input files must be sorted first for correct results.)

```
comm -12 file1.txt file2.txt
```

Suppresses columns 1 and 2, showing only the common lines between both files.

11. JOIN TWO FILES

Command: `join`

Work: Joins lines of two files based on a common field (like a database join), similar to SQL JOIN — both files must be sorted on the join field.

```
join file1.txt file2.txt
```

Joins lines from both files wherever the first field matches in both.

```
join -1 2 -2 1 file1.txt file2.txt
```

The `-1 2` and `-2 1` flags specify joining using field 2 of file1 and field 1 of file2 — useful when the matching column isn't the first field in both files.

12. THE `UNIQ` COMMAND

Command: `uniq`

Work: Removes (or reports) duplicate adjacent lines from a sorted file.

```
sort file.txt | uniq
```

Since `uniq` only catches adjacent duplicates, the file is typically sorted first, then piped into `uniq` to remove all duplicate lines.

```
uniq -c file.txt
```

The `-c` flag shows a count of how many times each line repeats, alongside the line itself.

```
uniq -d file.txt
```

The `-d` flag shows only the lines that had duplicates, printed once each.

13. THE TRANSFORMATION COMMAND (`TR`)

Command: `tr`

Work: Translates or deletes characters from input — works on a character-by-character basis, not whole words/lines.

```
tr 'a-z' 'A-Z' < file.txt
```

Converts all lowercase letters to uppercase in the file's content.

```
tr -d '0-9' < file.txt
```

The `-d` flag deletes all digit characters from the input.

```
tr -s ' ' < file.txt
```

The `-s` flag squeezes repeated consecutive spaces into a single space.